A. Comparison with Existing Compilers

To complement the preceding Hamiltonian-simulation benchmarks, we examine how our compiler compares with approaches that optimize lattice-surgery computations using different representations. These include the Lattice Surgery Compiler and libLSQECC [**?** **?**], resource allocation [**?**], code-substrate-based compilation in MQT QECC [**?**], and ZX-based compilation in TopoLS [**?**]. Dependency-aware compilation and compiler generation [**?** **?**], Sparo [**?**], and SAT-based synthesis of lattice-surgery subroutines [**?**] provide further complementary approaches. Their different instruction sets and architectural assumptions motivate a comparison using a common geometric representation. Our focus is automated compilation on explicitly specified patch resources, complementary to the workload-specific layouts used in bespoke resource estimates [**?** **?**].

We focus on TopoLS to relate ZX-level circuit optimization to bounded-depth spacetime routing. TopoLS simplifies a ZX graph before embedding it in three-dimensional spacetime, allowing spiders to merge and storage locations to be chosen during embedding. Our compiler optimizes spacetime paths on a regular patch layout that asymptotically reserves one in four sites for data storage and the remaining sites for routing. These optimizations act at different stages: ZX simplification changes the computational topology before placement, whereas double-slice routing controls the geometry and depth of individual paths. They are therefore complementary candidates for a combined compilation pipeline, with a lowering step from simplified ZX structures to the measurement primitives supported by the router. Both outputs can be represented as TQEC BlockGraphs [**?**], with TopoLS providing an official conversion, which enables the occupied geometries to be evaluated consistently.

We use the two six-qubit, 24-CNOT synthetic circuits shown in Figs. 1 and 2, generated with the max-parallel and min-parallel generators of MQT QECC [**?**]. These families follow the contrasting initial parallelism studied in the MQT QECC compilation benchmarks [**?** , Section VII.B], with the same gate-count scaling of four CNOTs per qubit. We use the generators' default random seed to obtain reproducible instances from an existing public implementation. The max-parallel generator forms groups of three disjoint CNOTs, while the min-parallel generator uses groups of two with enforced qubit overlap between successive groups. The latter is the smallest nonserial group size, retaining limited parallelism and providing a contrast in dependency structure at fixed circuit size.

For our compiler, we use SA-based mapping followed by double-slice routing with instruction look-ahead and kink-parity correction. For these CNOT-only circuits, the planar layout is a 7×7 grid with six data patches and 43 routing sites, obtained by removing the outer MSF ring. Six of the 49 sites hold data in this finite instance. TopoLS uses three input qubits per row, giving a 7×5 footprint, with the Python engine and default search settings, including ZX and direction optimization and a five-seed search. These settings allocate 49 sites to our compiler and 35 to TopoLS, with each compiler retaining its placement constraints and search strategy.

TABLE I: Comparison on the two six-qubit CNOT circuits. Sites denotes the available spatial allocation. TQEC's bounding-box dimensions (x, y, z) include input/output ports, with z denoting the time-coordinate extent. V is TQEC's spacetime volume, which sums occupied computation and storage cubes and excludes ports and pipes.

Circuit	Method	Sites	(x, y, z)	V
Max-parallel	Double-slice	49	$(7, 5, 13)$	178
Max-parallel	TopoLS	35	$(7, 5, 21)$	343
Min-parallel	Double-slice	49	$(5, 6, 13)$	166
Min-parallel	TopoLS	35	$(7, 5, 17)$	261

We evaluate both outputs using TQEC's official BlockGraph spacetime volume and bounding-box dimensions. For these circuits, the volume counts computation and storage cubes, with ports and pipes contributing zero volume. The bounding box includes input/output ports, and its z dimension measures the time-coordinate extent of the exported geometry. All exported graphs satisfy TQEC's local BlockGraph constraints and reproduce the input CNOT tensors up to an overall nonzero scalar within numerical precision. The occupied volumes of our exports equal the native scheduler's counts, preserving its storage and routing accounting. Figure 3 shows the resulting spacetime geometries, and Table I reports their dimensions and volumes alongside the available spatial allocations.

Double-slice routing yields occupied volumes of 178 and 166, compared with 343 and 261 for TopoLS, corresponding to reductions of approximately 48% and 36% on these instances. The exported geometries also have smaller time-coordinate extents of 13 for both circuits, compared with 21 and 17 for TopoLS. With the spatial allocations specified above, double-slice routing achieves more compact occupied geometry on both benchmark circuits. These results support bounded-depth routing as a useful optimization within an automated compilation pipeline and motivate its integration with ZX-level preprocessing.

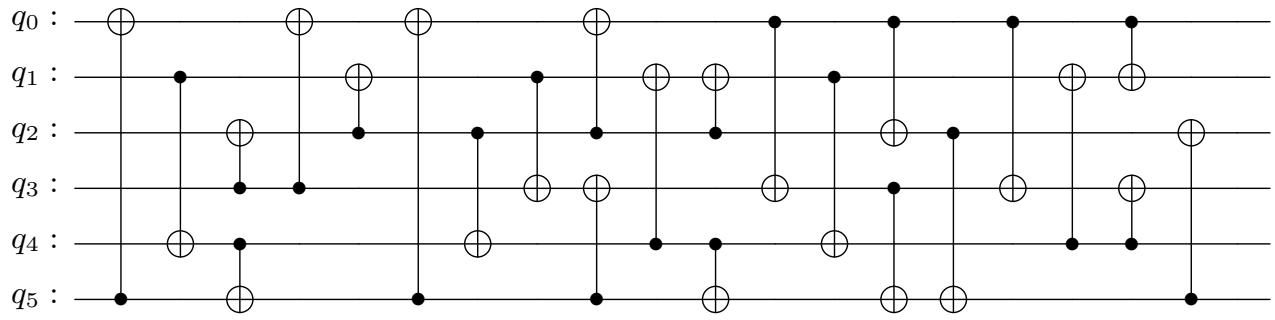


FIG. 1: Max-parallel benchmark circuit with six qubits and 24 CNOT instructions, generated with MQT QECC using its default random seed.

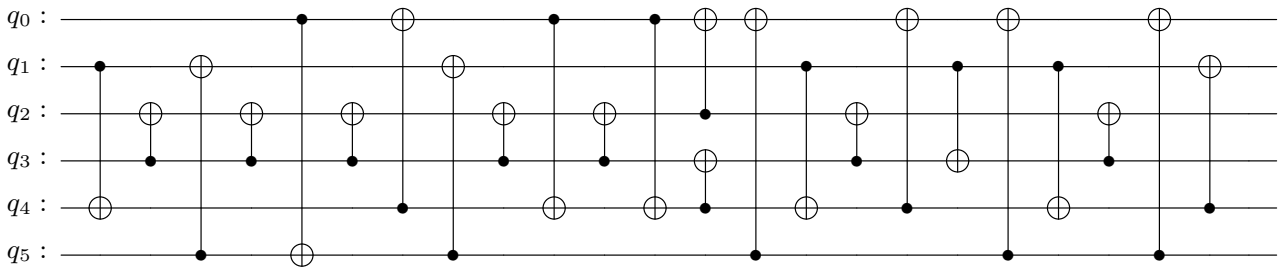
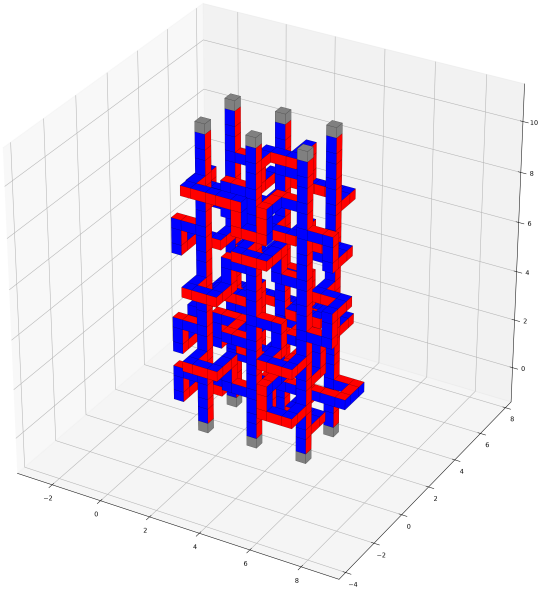
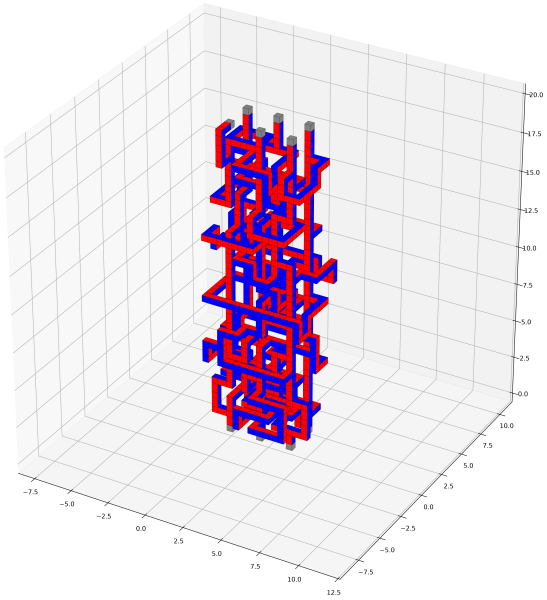


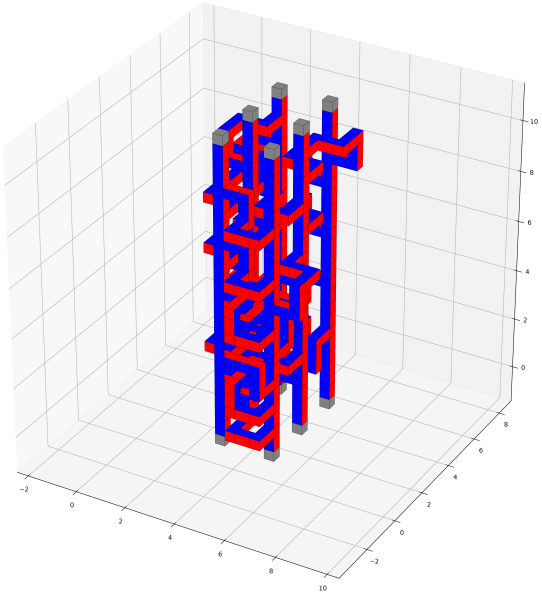
FIG. 2: Min-parallel benchmark circuit with six qubits and 24 CNOT instructions, generated with MQT QECC using its default random seed.



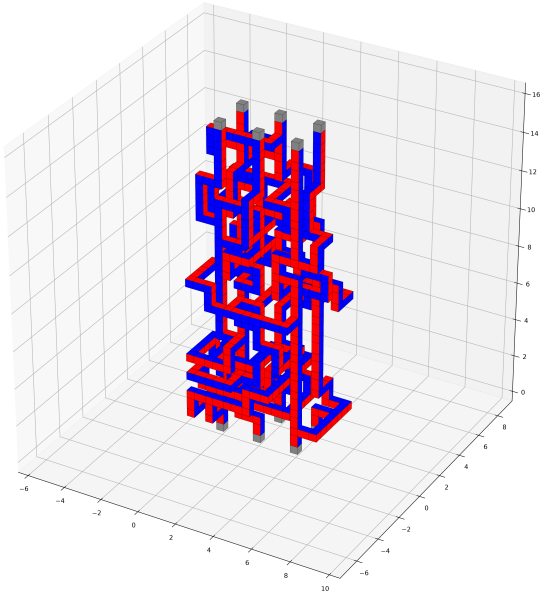
(a) Double-slice, max-parallel.



(b) TopoLS, max-parallel.



(c) Double-slice, min-parallel.



(d) TopoLS, min-parallel.

FIG. 3: Compiled spacetime geometries for the two benchmark circuits, rendered from TQEC BlockGraphs using TopoLS's renderer. The time direction is vertical, and blue and red faces indicate block boundary bases. Grey blocks indicate input/output ports and contribute zero volume. Each panel is independently fitted to its coordinate extent, while Table I provides the quantitative volume comparison.